"Sapienza" Università di Roma
Ingegneria dell'Informazione, Informatica e Statistica
Dipartimento di Informatica

Linguaggi e Compilatori

Autore
Vincenzo Bova

A.A. 2025/2026

Indice

1 Introduzione **2**
1.1 Linguaggi Compilati vs Linguaggi Interpretati 2
1.2 Struttura di un compilatore 2
1.3 Scoping 2
1.4 Passaggio dei parametri 3

1
Introduzione

1.1 Linguaggi Compilati vs Linguaggi Interpretati

In informatica moderna, quasi nessun linguaggio è puramente compilato oppure puramente interpretato. Il codice sorgente non viene mai passato direttamente all'hardware, ma esiste una gerarchia di componenti che collaborano tra loro:

- **Macchina del linguaggio (M_L):** è l'entità in grado di "capire" direttamente programmi in linguaggio L;
- **Trad(L):** prende il codice analizzato da M_L e lo traduce nel linguaggio I;
- **Macchina intermedia (M_I):** è l'entità in grado di "capire" il linguaggio intermedio I;
- **Int(I):** prende il codice analizzato da M_I e lo traduce nel linguaggio D;
- **Macchina di destinazione (M_D):** un tempo era la macchina fisica M_{HW} e D era il set di istruzioni della sua CPU. Con l'avvento dei Sistemi Operativi è diventata M_{SO} e D è composto da linguaggio macchina e chiamate di sistema.

Nella realtà il 99% dei linguaggi ad alto livello hanno M_I significative e i linguaggi puramente compilati o puramente interpretati sono dei casi limite.

1.2 Struttura di un compilatore

Un compilatore può essere suddiviso in due grandi blocchi:

- **Frontend:** si occupa di analizzare il codice sorgente e assicurarsi che sia corretto ed è composto da: *Lexical Analyzer*, *Syntax Analyzer* e *Semantic Analyzer*;
- **Backend:** si occupa di tradurre il programma analizzato in codice eseguibile ed è composto da: *Intermediate Code Generator*, *Machine-Independent Code Optimizer*, *Code Generator* e *Machine-Dependent Code Optimizer*.

In particolare, *Lexical* e *Syntax Analyzers* non lavorano quasi mai in sequenza lineare ma collaborano continuamente, mentre *Semantic Analyzer* e *Intermediate Code Generator* sono spesso fusi assieme. Ogni stadio della compilazione riceve e produce una rappresentazione intermedia, modificando la **Symbol Table**.

1.3 Scoping

Un compito fondamentale del compilatore è gestire la visibilità degli identificatori (variabili, funzioni, etc.) tramite le regole di **scoping**:

- **Scoping statico:** la visibilità di un nome è determinata al tempo di compilazione in base alla struttura del testo;
- **Scoping dinamico:** la visibilità dipende dall'ordine di chiamata delle funzioni a tempo di esecuzione.

1.4 Passaggio dei parametri

Determina come le informazioni vengono trasmesse tra il chiamante e la funzione:

- **Call by value:** viene passata una copia del valore. È sicuro ma può essere inefficiente per strutture grandi;
- **Call by reference:** viene passata la locazione di memoria. Permette effetti collaterali e risparmio di memoria;
- **Call by name:** l'espressione viene valutata solo quando effettivamente usata nella funzione (tipico dei linguaggi funzionali).